

LangGraph Core Concepts

A deep conceptual tour of LangGraph's building blocks: LLM workflows, graphs, nodes, edges, state, reducers, and the superstep execution model.

WHAT THIS CHAPTER COVERS

- ▶ What LangGraph is and why it exists
- ▶ LLM Workflows — definition and anatomy
- ▶ Five common LLM workflow patterns
- ▶ Graphs, Nodes & Edges in LangGraph
- ▶ State — shared mutable workflow memory
- ▶ Reducers — controlling state update strategy
- ▶ LangGraph Execution Model explained
- ▶ Supersteps and message passing

LangGraph Core Concepts

RECAP — CHAPTER 3

Chapter 3 asked the question: *why LangGraph when we already have LangChain?* It showed that LangChain’s chain-based model breaks down for complex workflows requiring loops, branching, parallelism, and shared state. LangGraph solves this by modelling any workflow as a **directed graph** built on top of LangChain’s ecosystem — complementing it rather than replacing it.

1. What Is LangGraph? (Quick Recap)

Before diving into individual concepts, a one-paragraph definition anchors everything that follows.

Definition

LangGraph is an **orchestration framework** for building intelligent, stateful, multi-step LLM workflows. It takes any LLM workflow you need to execute and **represents it as a graph**: a set of interconnected nodes where each node is a single task, and the edges between nodes encode the order and conditions of execution.

How it works in one sentence

You define a graph (nodes + edges + state), compile it once to validate its structure, then trigger it with an initial input — LangGraph automatically propagates execution through every node in the correct sequence, handling branching, parallelism, loops, and shared memory internally.

Core capabilities

Capability	What it enables
Graph representation	Any workflow expressed as nodes and edges
Parallel execution	Multiple nodes run simultaneously in one superstep
Loops / cycles	Flow can return to an earlier node
Conditional branching	Runtime routing between alternative paths
Shared state	All nodes read from and write to one state object
Memory / checkpointing	Conversation history and workflow state persist
Resumability	A failed workflow can be restarted from its last checkpoint

IMPORTANT

LangGraph is **not a replacement for LangChain**. It is built on top of LangChain and uses its models, prompts, tools, and retrievers. LangGraph adds the orchestration layer that LangChain's linear chain model cannot provide.

🔄 Quick Revision — LangGraph

- LangGraph is an orchestration framework that represents any LLM workflow as a directed graph.
- Each node in the graph is a Python function that performs one task; edges control execution order.
- Core features: parallel execution, loops, conditional branching, shared state, resumability.
- LangGraph builds on top of LangChain — it is complementary, not a replacement.

2. LLM Workflows

Definition

A *workflow* is a **series of tasks executed in order to achieve a goal**. An *LLM workflow* is any workflow where multiple tasks in that series depend on or invoke a large language model.

For example, an automated hiring workflow (write JD → post → shortlist → interview → onboard) becomes an LLM workflow the moment LLMs are used to draft the JD, evaluate CVs, or conduct the interview — because LLMs participate in the execution of individual tasks.

Anatomy of an LLM Workflow Step

Each step in an LLM workflow performs a **distinct, named task**. Common task types include:

- **Prompting** — sending an instruction to an LLM and receiving a response.
- **Reasoning** — asking the LLM to analyse, classify, or decide something.
- **Tool calling** — invoking an external API, database, or search engine.
- **Memory access** — reading or writing conversation history or a knowledge store.
- **Decision making** — routing or branching based on an LLM's output.

Workflow Topologies

LLM workflows are not always linear. The structure depends on how sub-tasks relate to each other. A task where each step depends on the previous must be linear; tasks that are mutually independent can run in parallel; a final decision that depends on runtime output requires branching; iterative refinement requires looping. LangGraph supports all four through its graph structure:

Topology	Description	Enabled by LangGraph
Linear	Steps execute one after another in a fixed sequence	Sequential edges

Parallel	Multiple steps execute simultaneously	Fan-out edges + supersteps
Branching	Runtime routing between two or more alternative paths	Conditional edges
Looped	Flow returns to an earlier step until a condition is met	Cycle edges

REAL WORLD INSIGHT

In production AI systems, most sophisticated applications combine more than one topology. A customer-support agent might route incoming queries (branching), then answer each with an LLM (linear), while a quality-checker runs in parallel, and poor answers loop back for revision. LangGraph is designed to express exactly this kind of compound topology.

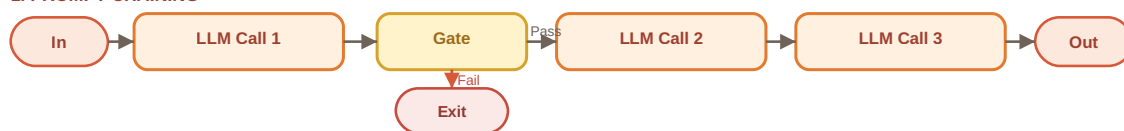
🔄 Quick Revision — LLM Workflows

- A workflow is a sequence of tasks executed to achieve a goal; an LLM workflow uses LLMs in one or more of those tasks.
- Step types: prompting, reasoning, tool calling, memory access, decision making.
- Topologies: linear, parallel, branching, looped — all expressible in LangGraph.
- Each workflow is application-specific, but certain patterns recur across many applications.

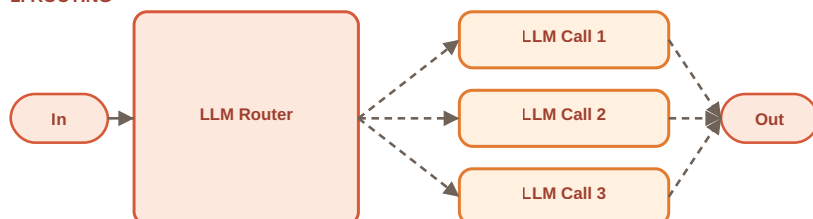
3. Five Common LLM Workflow Patterns

While every application has a unique workflow, five **structural patterns** appear repeatedly across different domains. Recognising them in a new problem is the first step to choosing the right LangGraph design.

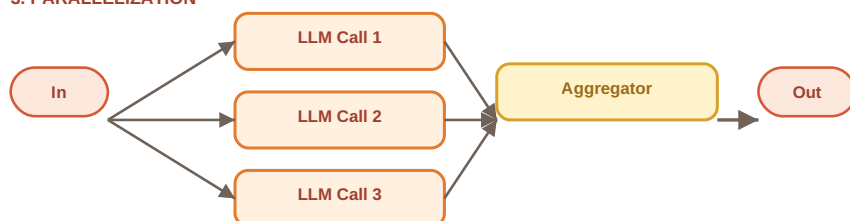
1. PROMPT CHAINING



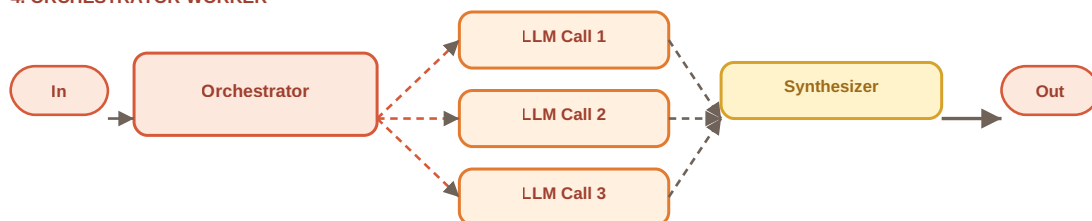
2. ROUTING



3. PARALLELIZATION



4. ORCHESTRATOR-WORKER



5. EVALUATOR-OPTIMIZER

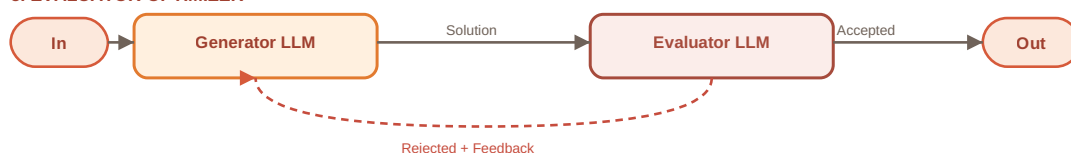


Figure 4.1 — The five common LLM workflow patterns. Each pattern solves a distinct orchestration problem.

1. Prompt Chaining

The task is broken into a **sequence of LLM calls** where the output of one call becomes the input of the next. A gate (quality check) may be inserted between calls to validate intermediate output and short-circuit if quality is insufficient.

Example: Generate a report outline with LLM 1, then expand it into a full report with LLM 2. A gate between them checks: “is this outline shorter than 500 words?” If not, exit early.

Use when: a complex task can be meaningfully decomposed into serial sub-tasks, and intermediate checkpoints add reliability.

2. Routing

A **router LLM** analyses an incoming request and dispatches it to the most appropriate specialist LLM. The router acts as a decision-maker; it does not generate the final answer itself.

Example: A customer-support chatbot routes refund queries to the Refund LLM, technical questions to the Technical LLM, and sales enquiries to the Sales LLM.

Use when: different query types require different specialised prompts, knowledge, or models.

3. Parallelization

A task is decomposed into **independent sub-tasks that run simultaneously**. Results are collected and aggregated by a final node.

Example: Content moderation on a YouTube video runs three parallel checks simultaneously — community guidelines, misinformation detection, and sexual content — then an aggregator decides whether to publish.

Use when: sub-tasks are mutually independent and can be evaluated concurrently to reduce total latency.

IMPORTANT

Parallelization vs. Orchestrator-Worker: In parallelization the sub-tasks are *predefined* — you know upfront what each parallel LLM will check. In the Orchestrator-Worker pattern the sub-tasks are *dynamically assigned* at runtime based on the incoming query.

4. Orchestrator-Worker

An **orchestrator LLM** receives the input and dynamically decides what sub-tasks need to be done and which worker LLMs should handle them. Workers execute in parallel and report results to a synthesizer.

Example: A research assistant that must produce a report on a query. The orchestrator analyses the query: if it is scientific, it directs Worker 1 to search Google Scholar; if it is a news event, it directs Worker 2 to search Google News. The specific assignments vary per query.

Use when: sub-task nature is not known in advance and must be decided by reasoning over the input.

5. Evaluator-Optimizer

Two LLMs operate in a **feedback loop**: a Generator produces a candidate solution, and an Evaluator scores it against defined criteria. If rejected, the Evaluator's feedback is fed back to the Generator for the next iteration. The loop continues until the Evaluator accepts the solution.

Example: Drafting an email. The Generator writes a draft; the Evaluator checks tone, length, and clarity. If any criterion fails, feedback is sent back and the Generator revises. After several rounds, the accepted draft is returned.

Use when: the task is inherently iterative and quality can be assessed against explicit criteria (creative writing, code generation, document drafting).

INTERVIEW TIP

When asked to design an LLM system, first identify which of these five patterns fits the problem. Interviewers look for this kind of structured thinking. Bonus: explain that in practice a single production system often combines two or more patterns — for example, routing followed by parallelization within each specialist branch.

🔄 Quick Revision — Five Workflow Patterns

- **Prompt Chaining:** serial LLM calls where each output feeds the next, with optional quality gates.
- **Routing:** a router LLM classifies the input and dispatches to the right specialist LLM.
- **Parallelization:** predefined independent sub-tasks run simultaneously; results aggregated.
- **Orchestrator-Worker:** sub-tasks are dynamically assigned at runtime based on the query.
- **Evaluator-Optimizer:** Generator + Evaluator loop until the solution meets defined criteria.

4. Graphs, Nodes & Edges

Graphs, nodes, and edges are collectively the **most important core concept in LangGraph**. Everything else — state, reducers, the execution model — exists in service of this fundamental representation.

Why a Graph?

Any LLM workflow can be visualised as a flowchart: boxes for tasks, arrows between them. LangGraph formalises this intuition. When you give LangGraph a workflow to execute, it **represents that workflow as a directed graph** and executes the graph node-by-node, edge-by-edge.

The graph structure is powerful because it can natively express every topology discussed in Section 2: linear sequences, parallel branches, conditional forks, and cycles — all within the same unified abstraction.

Nodes

A *node* represents a **single task** in the workflow. Behind the scenes, every node in LangGraph is simply a **Python function**.

- Each node receives the current **state** as input.
- It performs its task (call an LLM, invoke a tool, make a decision, etc.).
- It updates the state with its result and passes the updated state forward.

If you can write a Python function, you can create a LangGraph node. There is no special class to inherit or interface to implement.

Edges

An *edge* connects two nodes and encodes **when to execute the next node**. Edges answer the question: “*after this node finishes, where does execution go next?*”

Edge type	Behaviour	Workflow pattern
Sequential	Always proceeds to one fixed next node	Prompt chaining, linear flows
Parallel	Fans out to multiple next nodes simultaneously	Parallelization, orchestrator-worker
Conditional	Routes to one of several next nodes based on runtime state	Routing, branching, evaluator-optimizer
Cycle (loop)	Returns flow to an earlier node	Evaluator-optimizer, retry loops

IMPORTANT

Nodes tell you *what to do*; edges tell you *when to do it*. This separation is the key design principle: task logic lives entirely in nodes, while execution order lives entirely in edges. Keeping these concerns separate makes complex workflows manageable.

The UPSC Essay Evaluator — A Worked Example

The following workflow helps UPSC aspirants practise essay writing. It demonstrates how a real-world requirement maps naturally onto a LangGraph graph with multiple node types, parallel evaluation, conditional routing, and a retry loop.

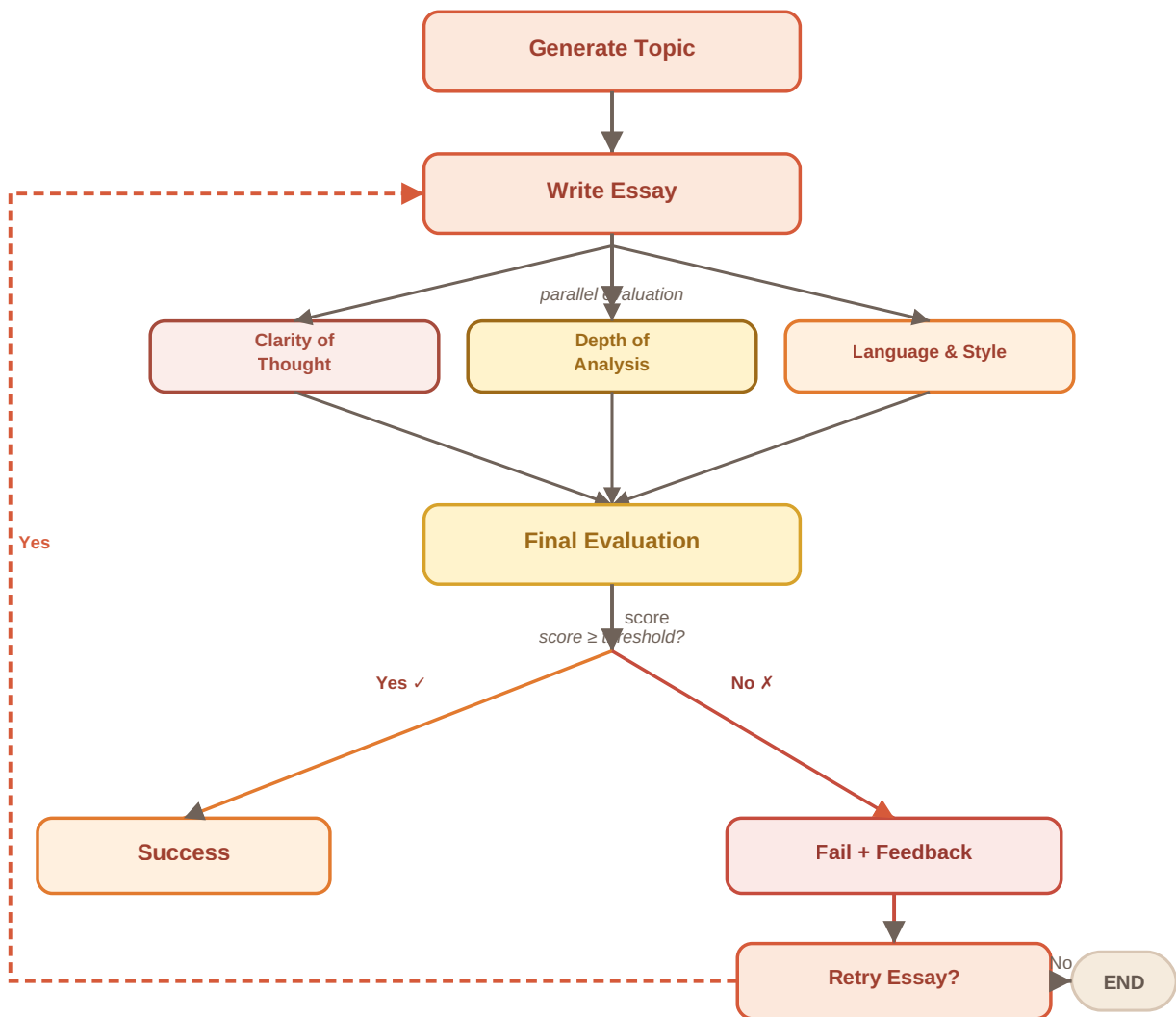


Figure 4.2 — UPSC Essay Evaluator workflow as a LangGraph graph. Nodes = Python functions; dashed loop = conditional cycle back to Write Essay.

The workflow steps are: generate a topic → student writes essay → three evaluators run in parallel (Clarity of Thought, Depth of Analysis, Language & Style, each scoring out of 5) → scores aggregated into a final score out of 15 → conditional branch: if score ≥ 10 the student is congratulated (Success); if score < 10 feedback is given and the student may choose to revise and retry (loop) or exit.

REAL WORLD INSIGHT

This single workflow combines *parallelization* (three evaluators run simultaneously), *conditional routing* (success vs. fail branch), and an *evaluator-optimizer loop* (retry with feedback). In practice, most sophisticated LangGraph applications are compositions of these patterns rather than pure instances of a single pattern.

🔄 Quick Revision — Graphs, Nodes & Edges

- LangGraph represents any LLM workflow as a directed graph of nodes and edges.
- A node = one task = one Python function. No special class required.
- Each node receives the full state, updates it, and passes it forward via edges.
- Edge types: sequential, parallel (fan-out), conditional, and cycle (loop).
- Nodes answer *what to do*; edges answer *when to do it*.

5. State

Definition

State in LangGraph is the **shared memory that flows through your entire workflow**. It holds all the data that nodes produce and consume as the graph executes.

Formally: state is a **TypedDict** (a typed Python dictionary subclass) that you define before creating the graph. It is passed as input to every node, and every node may update it with its results.

Why State Is Needed

Any non-trivial workflow needs data to flow from one step to the next. Without a shared state object, each node would need explicit function arguments linking it to the previous node — which becomes unmanageable as the graph grows. State solves this by giving all nodes a **single shared data container**.

Two Critical Properties of State

- **Shared** — every node in the graph has access to the complete state, regardless of its position in the graph. A node added later can read data written by any earlier node.
- **Mutable** — any node can update any key in the state. As execution progresses, the state evolves: keys are written, overwritten, or appended to.

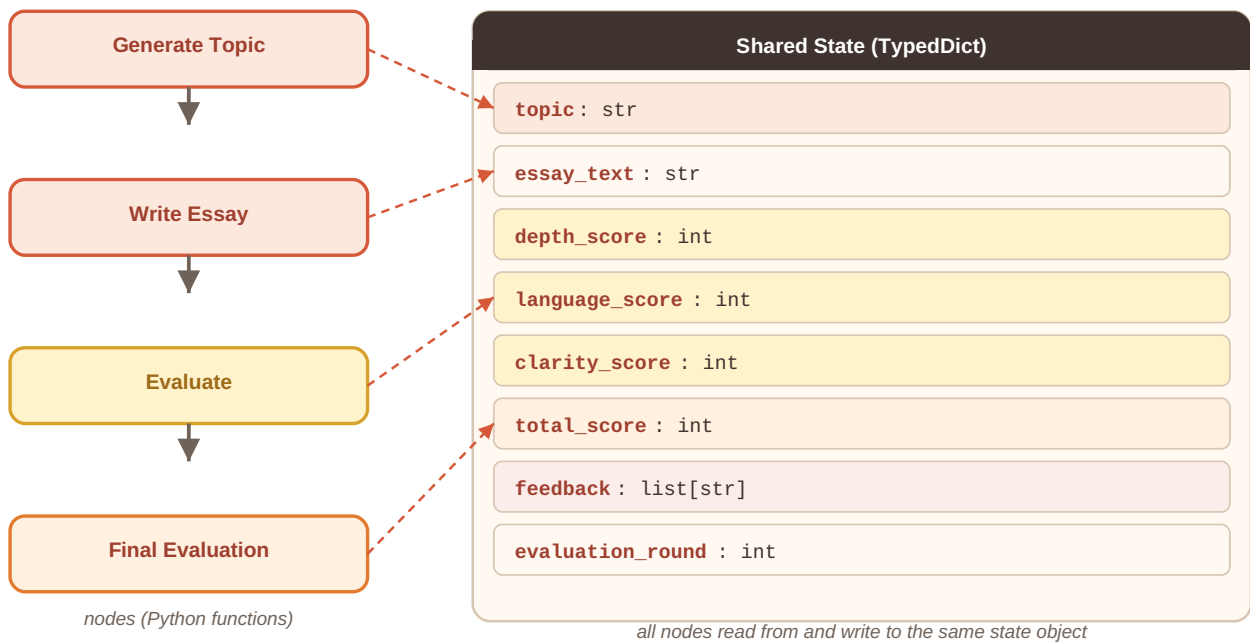


Figure 4.3 — State is a shared TypedDict that flows through all nodes. Each node reads from and writes to the same state object.

Designing a State Schema

Before building any LangGraph workflow, you must design its state schema — the complete list of key–value pairs the workflow will need. For the UPSC Essay Evaluator example, the state contains:

```
from typing import TypedDict, Annotated
import operator

class EssayState(TypedDict):
    topic: str # essay topic generated at the start
    essay_text: str # student's submitted essay
    depth_score: int # score for depth of analysis (0-5)
    language_score: int # score for language quality (0-5)
    clarity_score: int # score for clarity of thought (0-5)
    total_score: int # aggregated final score (0-15)
    feedback: Annotated[list[str], operator.add] # accumulated feedback items
    evaluation_round: int # how many revision attempts so far
```

Notice that `feedback` uses `Annotated[list[str], operator.add]`. This is a **reducer annotation** — it tells LangGraph to *append* new feedback items rather than overwrite the list. This is covered in detail in Section 6.

BEST PRACTICE

Design your state schema *before* you write any node. List every piece of data that any node will need to read or write during execution. Treat state design as you would a database schema: getting it right upfront prevents painful refactoring later.

COMMON MISTAKE

Storing data *inside* a node function as a local variable when it needs to be accessed by a later node. Local variables disappear when the function returns. Any data that must survive between nodes **must live in the state**.

🔄 Quick Revision — State

- State is a TypedDict (typed Python dict) that is shared across all nodes in the graph.
- It is **shared** (every node can read all keys) and **mutable** (any node can update it).
- Design the full state schema before writing any node — list every data point the workflow needs.
- State evolves continuously: nodes read the current state, perform their task, then write updates back.
- Any data needed by more than one node must be stored in state, not in local variables.

6. Reducers

Definition

A *reducer* defines **how updates from nodes are applied to the shared state**. Specifically, when a node writes a new value to a state key, the reducer for that key determines whether the new value *replaces* the existing value, *appends* to it, or is *merged* with it.

Each key in the state can have its own reducer. If no reducer is specified for a key, LangGraph applies the default behaviour: **replace** (the old value is overwritten by the new one).

Why Reducers Are Needed

The default replace behaviour works perfectly for most data. But in two important scenarios it causes problems:

SCENARIO A: CHATBOT MESSAGE HISTORY

A chatbot stores conversation messages in a `messages` key. With the default replace strategy, when the LLM node writes a reply, it overwrites the entire message list with just the latest message — meaning the LLM node on the next turn has no memory of previous exchanges. The correct behaviour is to *append* each new message to the list.

Turn	Replace (broken)	Add reducer (correct)
1: User says “Hi, I’m Nitesh”	<code>messages = [“Hi, I’m Nitesh”]</code>	<code>messages = [“Hi, I’m Nitesh”]</code>
2: LLM replies “Hi! How can I help?”	<code>messages = [“Hi! How can I help?”]</code> — name lost	<code>messages = [“Hi, I’m Nitesh”, “Hi! How can I help?”]</code>
3: User asks “What’s my name?”	LLM cannot answer — name was erased	LLM answers correctly — full history preserved

SCENARIO B: TRACKING ESSAY REVISIONS

When a student retries the UPSC essay, the new essay should be stored alongside — not instead of — previous attempts, so the student can see their progression. Using the add reducer on `essay_text` (if we wanted version history) achieves this.

How to Specify a Reducer

Reducers are specified as type annotations on state keys using `Annotated`:

```
from typing import Annotated
import operator

class ChatState(TypedDict):
    # Default: replace. Each node call overwrites this.
    current_topic: str

    # Add reducer: new messages are appended to the list.
    messages: Annotated[list[str], operator.add]

    # Custom reducer function is also possible:
    # my_key: Annotated[list, my_custom_merge_fn]
```

The second argument to `Annotated` is the reducer function. `LangGraph` calls this function with `(existing_value, new_value)` whenever a node updates that key. `operator.add` on lists produces list concatenation, which is the append behaviour we want.

Reducer Types at a Glance

Strategy	Syntax	Use when
Replace (default)	No annotation needed	The key holds a single current value that should be overwritten (scores, booleans, current topics)
Append (add)	<code>Annotated[list[T], operator.add]</code>	The key accumulates a list over time (messages, feedback items, essay versions)
Custom merge	<code>Annotated[T, my_fn]</code>	Domain-specific merge logic (e.g. dict merging, deduplication)

BEST PRACTICE

Always use the `operator.add` reducer on any state key that holds a list of messages, events, or feedback items that must accumulate over the lifetime of the workflow. Forgetting to add the reducer on message history is one of the most common bugs in early `LangGraph` implementations.

INTERVIEW TIP

If asked “*how does LangGraph handle shared state when multiple nodes run in parallel?*” — answer with reducers. Parallel nodes may attempt to write to the same key simultaneously. LangGraph uses each key’s reducer to merge the concurrent writes into a single consistent state value. This is the primary reason reducers matter in parallelization scenarios.

🔄 Quick Revision — Reducers

- A reducer defines how a node’s write to a state key is applied: replace, append, or custom merge.
- Default (no annotation) = replace: the new value overwrites the old one entirely.
- Add reducer (`Annotated[list[T], operator.add]`) = append: new items are added to the list.
- Reducers are critical in parallelization: they merge concurrent writes from parallel nodes.
- Always annotate message history and accumulating lists with `operator.add`.

7. LangGraph Execution Model

Understanding how LangGraph actually executes a workflow behind the scenes removes the “black-box” feeling and makes debugging and performance optimisation much easier. LangGraph’s execution model is inspired by **Google Pregel** — a large-scale graph-processing system used inside Google products — adapted for LLM workflows.

The Four Phases

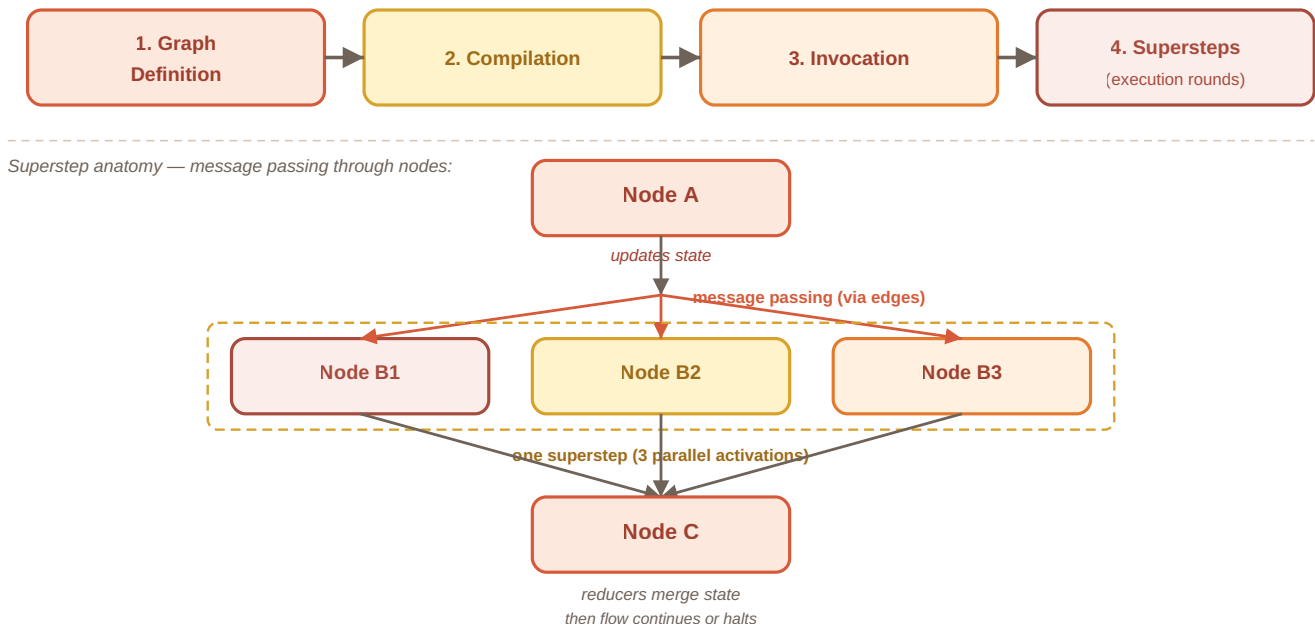


Figure 4.4 — LangGraph execution model: four phases, plus the anatomy of a single superstep with parallel node activations.

PHASE 1 — GRAPH DEFINITION

You define the graph by specifying three things: the **state schema** (a TypedDict), the **nodes** (Python functions), and the **edges** (connections between nodes, including conditional edges). This is pure Python — no execution happens yet.

PHASE 2 — COMPILATION

You call `.compile()` on the `StateGraph` object. LangGraph validates the graph structure: it checks for disconnected (orphaned) nodes, verifies that every conditional edge has a valid target for each branch, and prepares the internal execution plan. This step exists to catch structural errors early — before any LLM calls are made.

PHASE 3 — INVOCATION

You call `.invoke(initial_state)` on the compiled graph. LangGraph sends the initial state as the first **message** to the entry node. This triggers the first node activation.

PHASE 4 — SUPERSTEPS (EXECUTION ROUNDS)

Execution proceeds in **rounds called supersteps**. In each superstep:

1. One or more nodes receive the current state as a message through their incoming edges and become **active**.
2. Each active node executes its Python function and produces a **partial state update**.
3. Updates are merged into the shared state using each key's reducer.
4. The updated state is sent via edges as a message to the next set of nodes.
5. If those next nodes receive a message, they become active — starting the next superstep.

Execution halts when no nodes are active and no messages are in transit through any edge. This is the natural end of the workflow.

Message Passing

The mechanism by which state is sent from one node to the next through an edge is called *message passing*. You never call nodes manually. You give the entry node an initial state and message passing handles everything else: each node's output automatically becomes the input to the next node(s) as determined by the edge configuration.

Why “Superstep” and Not Just “Step”?

A conventional “step” implies a single unit of work. But because LangGraph supports parallel fan-out, a single round of execution may activate *multiple nodes at once*. The term **superstep** acknowledges this: one superstep can contain one sequential activation *or* many parallel activations. The number of node executions per superstep depends entirely on the graph structure at that point.

REAL WORLD INSIGHT

The Google Pregel inspiration is significant. Pregel was designed to process graphs with billions of nodes. By adopting its superstep model, LangGraph gains a clean abstraction for parallelism: within a superstep, all active nodes can run truly concurrently without needing explicit thread management from the developer.

INTERVIEW TIP

When asked how LangGraph executes a workflow, walk through the four phases in order: Graph Definition → Compilation → Invocation → Supersteps. Then explain message passing in one sentence: *“Each node receives the current state, updates it, and sends the updated state forward through edges to the next node(s).”* This answer demonstrates a precise understanding of the framework.

🔄 Quick Revision — Execution Model

- Phase 1 — Graph Definition: define state schema, nodes, and edges.
- Phase 2 — Compilation: `.compile()` validates graph structure and catches orphaned nodes.
- Phase 3 — Invocation: `.invoke(initial_state)` sends the initial state to the entry node.
- Phase 4 — Supersteps: rounds of execution; each round activates nodes, runs them, and propagates updated state via message passing.
- Execution halts when no nodes are active and no messages are in transit.
- A superstep may contain 1 or many parallel node activations — hence “super”step.
- Message passing = sending updated state through edges to the next node(s) automatically.

K. Key Takeaways

- **LangGraph** is an orchestration framework that represents any LLM workflow as a directed graph of nodes and edges, built on top of LangChain.
- **LLM Workflows** are sequences of tasks that use LLMs at one or more steps; they can be linear, parallel, branching, or looped.
- **Five Common Patterns** — Prompt Chaining, Routing, Parallelization, Orchestrator-Worker, and Evaluator-Optimizer — recur across most LLM applications.
- **Nodes** are Python functions that each represent one task; **Edges** control execution order and can be sequential, parallel, conditional, or cyclic.
- **State** is a shared TypedDict accessible and mutable by every node; it holds all the data a workflow needs throughout its execution lifecycle.
- **Reducers** define how node writes are applied to state: replace (default), append (`operator.add`), or custom merge — critical for parallel workflows and message history.
- **Execution Model** follows four phases: Graph Definition → Compilation → Invocation → Supersteps; nodes are activated via message passing and execution halts when no messages remain in transit.
- **Supersteps** are execution rounds that may contain one or many parallel node activations; the term reflects that a single round can be more than one sequential step.

R. Revision Sheet

LangGraph

An orchestration framework that represents any LLM workflow as a directed graph, providing parallel execution, loops, branching, shared state, and resumability.

LLM Workflow

A sequence of tasks where at least one step invokes an LLM; can be linear, parallel, branching, or looped.

Prompt Chaining

Serial LLM calls where each output feeds the next, with optional quality gates to validate intermediate results.

Routing

A router LLM classifies the input and dispatches to the most appropriate specialist LLM.

Parallelization

Predefined independent sub-tasks run simultaneously; results are collected by an aggregator node.

Orchestrator-Worker

An orchestrator LLM dynamically assigns sub-tasks to worker LLMs at runtime based on the input query.

Evaluator-Optimizer

Generator + Evaluator loop: Generator produces a solution; Evaluator scores it; if rejected, feedback triggers a new generation attempt.

Superstep

One execution round in LangGraph. May activate one or many nodes in parallel. Execution halts when no active nodes and no in-transit messages remain.

Node

A Python function that performs one task, receives the shared state as input, and writes a partial update back to state.

Message Passing

The mechanism by which updated state is sent through edges from one node to the next, automatically, without manual node calls.

Edge

A directed connection between nodes that controls execution order. Types: sequential, parallel (fan-out), conditional, and cycle.

Compilation

Calling `.compile()` on a StateGraph to validate structure (detect orphaned nodes, check conditional edges) before execution begins.

State

A TypedDict shared across all nodes. Holds all workflow data. Is mutable — any node can update any key.

Reducer

Defines how a node's write is applied to a state key: replace (default), add/append, or custom merge function.

I. Interview Questions

Q1. What is LangGraph and how does it differ from LangChain? Is it a replacement?

Q2. Define an LLM workflow. What makes a workflow an “LLM workflow” specifically?

Q3. Explain the five common LLM workflow patterns and give a real-world use case for each.

Q4. What is the difference between Parallelization and Orchestrator-Worker patterns?

Q5. In LangGraph, what is a node and what is an edge? What does each represent at the code level?

Q6. What is LangGraph State? Why must workflow data be stored in state rather than in local node variables?

Q7. What is a reducer in LangGraph? When would you use `Annotated[list[str], operator.add]` instead of the default replace behaviour?

Q8. Explain the four phases of LangGraph’s execution model. What does `.compile()` actually do?

Q9. What is a superstep? Why is the term “superstep” used instead of “step”?

Q10. How does LangGraph handle concurrent writes to the same state key when multiple nodes run in parallel?

F. Further Reading

- LangGraph Official Documentation — Concepts: langgraph.readthedocs.io (covers StateGraph, nodes, edges, and checkpoints in depth)
- LangGraph Conceptual Guides — “LangGraph Glossary” (official definitions of state, reducer, superstep, message passing)
- Google Pregel Paper — “Pregel: A System for Large-Scale Graph Processing” (Malewicz et al., SIGMOD 2010) — the academic foundation for the superstep model
- LangChain Blog — “Introducing LangGraph” (motivation and design decisions behind the framework)
- Anthropic — “Building Effective Agents” (covers agentic patterns including orchestrator-worker and evaluator-optimizer)
- Python typing module documentation — `TypedDict` and `Annotated` (standard library reference for state schema construction)